

University of Seoul

Seoul Campus

School of Electrical and Computer Engineering



Lab 4: Stm32 Dual Elevator Embedded System

Subject:

Embedded Systems Applications Design

Student:

Roberto Raymundo Gmez Vargas
2025020338

Professor:

Ye Gu Kang

Seoul, South Korea, December 2025

Abstract

This project presents the design and implementation of a dual-elevator control system using the STM32 NUCLEO-G474RE development board. The system simulates a five-floor building with two independent elevators driven by continuous rotation servo motors. User interaction is performed through a 4x4 matrix keypad, which allows floor selection and elevator calling.

Magnetic limit sensors and permanent magnets are used to detect reference positions and motion limits, enabling safe operation and floor positioning. A scheduling strategy based on a collectiveselective algorithm is implemented to optimize elevator movement and reduce unnecessary travel. The system processes multiple requests and determines an efficient order for servicing floor calls.

Real-time system activity, including elevator position, movement direction, and active requests, is transmitted through UART communication and monitored using a computer terminal interface. The project integrates hardware design, embedded programming, and system logic to demonstrate a complete embedded control application with real-time feedback, safety mechanisms, and optimized scheduling.

Keywords: Embedded systems, Elevator control system, STM32, UART communication, Matrix keypad, Hall effect sensors, Scheduling algorithm, Real-time monitoring

Contents

1	Project Overview	6
1.1	Introduction	6
1.2	Final Objective of the Project	6
1.3	Project Description and Scope	7
2	Project Necessity	8
2.1	Importance of Automation Systems	8
2.2	Educational Motivation	8
2.3	Applications Similar to the Proposed System	9
2.4	Comparison with Existing Systems	9
2.5	Current Challenges and Project Significance	9
3	Execution Strategy	10
3.1	Project Strategy and Methods	10
3.2	Project Implementation Plan	11
3.3	Project Duration and Cost Justification	11
4	Project Performers Competencies	12
4.1	Technical Background	12
4.2	Hardware and Embedded System Skills	12
4.3	Mechanical Design and Fabrication	13
4.4	Software Design and Problem Solving	13
4.5	Capability for Project Completion	13
5	Utilization Plan and Expected Outcomes	14

5.1	Project Utilization Plan	14
5.2	Expected Outcomes and Impacts	15
5.2.1	Scientific and Technological Impact	15
5.2.2	Economic and Industrial Perspective	15
5.2.3	Social and Educational Impact	15
5.2.4	Differentiation from Existing Solutions	15
6	System Framework	16
6.1	Overall System Architecture	16
6.2	System Workflow	17
6.3	System Monitoring via UART	18
6.4	Summary of System Behavior	18
7	Inputs and Outputs	19
7.1	Input Section Specification	19
7.1.1	Purpose of Input	19
7.1.2	Rules (Input Interpretation and Flow)	19
7.1.3	Required Components	20
7.1.4	Input Circuit	20
7.1.5	Description of Input Components	20
7.1.6	Input Processing Method	20
7.2	Output Section Specification	21
7.2.1	Purpose of Output	21
7.2.2	Rules (Output Decision Logic)	21
7.2.3	Required Components	21
7.2.4	Output Circuit	22
7.2.5	Description of Output Components	22
7.2.6	Output Processing Method	22
8	Phase 1; Mechanical Design	23
8.1	Elevator Box Design	23
8.2	Supporting Structure	24

8.3	Base and Rail System	25
8.4	Final Assembly	26
9	Phase 2; Hardware Integration	27
9.1	System Overview	29
9.2	Microcontroller and Power Distribution	29
9.3	Servo Motor Connections	29
9.4	Matrix Keypad Interface	30
9.5	CubeIDE Pin Configuration	30
9.6	Physical Assembly	31
9.7	Demonstration Video	31
10	Phase 3; Code and Implementation of Scheduling Logic	32
10.1	Software Architecture Overview	32
10.2	Scheduling Algorithm Used	33
10.3	Code Explanation	33
10.4	Includes and Definitions	33
10.4.1	Keypad GPIO Definitions and Timing Constants	34
10.5	Keypad Scanning Logic	35
10.6	Servo Motor Control and Per-Servo Calibration	36
10.6.1	Low-Level PWM Write Function	36
10.6.2	Motivation for Per-Servo Profiles	37
10.6.3	Deadband Filtering	37
10.6.4	Elevator PWM Mapping and High-Level Commands	38
10.7	Data Structures for Requests and Elevator State	39
10.8	Mapping Keypad Input to Floors	41
10.9	Request Queue Implementation	42
10.10	Elevator Selection (Greedy Scoring Function)	43
10.11	State Machine for Elevator Motion	44
10.11.1	Starting a Motion Segment	44
10.11.2	Assigning Requests to Idle Elevators	45

10.11.3 Periodic Elevator Update	46
10.12 Main Program Flow	48
10.12.1 Initialization and PWM Startup	48
10.12.2 Main Loop Execution	49
10.13 Timer Configuration Rationale	50
11 Phase 4; Results	51
12 Conclusion	52
Bibliography	53

List of Figures

6.1	Input system	16
8.1	Elevator box model designed in SolidWorks	24
8.2	Isometric view of the complete elevator shaft, showing the truss reinforcement pattern.	25
8.3	Top view of the base structure with integrated reinforcement elements.	25
8.4	Final assembly of the mechanical elevator structure.	26
9.1	Electrical connections.	27
9.2	Electrical connections	28
9.3	Electrical connections	28
9.4	CubeIDE Pin Configuration	30
10.1	Input system	41
10.2	Timer Configuration	50

Chapter 1

Project Overview

1.1 Introduction

Vertical transportation systems such as elevators are essential components in modern infrastructure, requiring reliable control, safety mechanisms, and efficient scheduling strategies. Although full-scale elevator systems are complex and expensive, scaled models provide an effective platform to study their operation principles, control logic, and embedded system integration.

This project focuses on the design and implementation of a five-floor dual-elevator system using an embedded microcontroller platform. The system integrates mechanical components with electronic hardware and real-time software to simulate typical elevator functions, including floor selection, movement control, position detection, and operational feedback.

Using the STM32 NUCLEO-G474RE as the main controller, the project demonstrates how embedded systems can be used to manage multiple actuators, communicate with external interfaces, and execute task scheduling algorithms. The project combines concepts from digital electronics, control logic, communication protocols, and system integration, providing a practical learning experience in embedded systems engineering.

1.2 Final Objective of the Project

The final objective of this project is to design, build, and validate a fully functional dual-elevator embedded control system capable of operating across five floors. The system must correctly process user requests, determine elevator movement strategies, detect motion limits, and display real-time system activity through serial communication.

The project aims to simulate intelligent elevator behavior by implementing realistic control logic and scheduling techniques. In addition to demonstrating system operation, the project emphasizes reliability, safety, and efficient task management. The ultimate goal is to integrate mechanical design, electronics, and software into a real-time automated system that reflects the principles used in real-world elevator control applications.

1.3 Project Description and Scope

This project involves the complete development of a scaled elevator automation system including mechanical construction, electronic interfacing, and embedded software implementation. Two independent elevators operate inside a five-floor structure. Each elevator is driven by a continuous-rotation servo motor connected to a rope-and-drum mechanism that allows vertical motion.

User interaction is provided through a 4x4 matrix keypad used for inputting destination floors and elevator calls. Magnetic sensors and permanent magnets are installed at the motion boundaries to detect top and bottom limits, preventing overtravel and ensuring safe operation. These sensors serve as reference points for elevator control logic.

The software is implemented in the C programming language using the STM32 HAL library. A collectiveselective scheduling algorithm determines the floor service order based on current direction and active requests. The algorithm allows simultaneous management of two elevators while minimizing unnecessary movement.

Real-time system information including elevator position, active requests, and direction of movement is transmitted via UART communication and displayed on a computer terminal. This monitoring system provides visibility into system behavior and assists with debugging, testing, and demonstration.

The scope of the project includes all major subsystems of an automated control application: user input, actuator control, position sensing, communication, and scheduling logic. Emphasis is placed on functional correctness, safety handling, and modular software structure rather than industrial-grade mechanical complexity.

Chapter 2

Project Necessity

2.1 Importance of Automation Systems

Automation systems play a key role in modern infrastructure, especially in applications that require safe, reliable, and efficient operation. Elevator systems are a common example of automation in everyday life, as they operate continuously in residential buildings, commercial centers, and hospitals. These systems must respond to user input, control physical motion, and enforce safety constraints in real time.

Understanding the internal operation of such systems is essential for embedded systems engineers. An elevator system combines sensing, actuation, communication, and decision-making into one application, making it an ideal learning platform for developing real-world engineering skills.

2.2 Educational Motivation

One of the main motivations for this project is to transform theoretical concepts into a practical implementation. Concepts such as GPIO control, timers, interrupts, and serial communication are often taught separately, but real engineering systems require all these elements to work together.

This project forces the integration of hardware and software into a single automated system. By designing, building, and programming a multi-elevator platform, experience in debugging, testing, system coordination, and performance optimization will be developed.

2.3 Applications Similar to the Proposed System

Systems similar to the one developed in this project are widely used in industry. Automated vertical transportation systems exist not only in buildings but also in storage systems, robotic platforms, and logistics centers. In manufacturing environments, machines often rely on scheduling algorithms to coordinate operations efficiently.

Beyond elevators, scheduling and automation principles apply to traffic systems, robotics, warehouse automation, and industrial conveyor belts. Knowledge gained from this project is therefore transferable to many engineering fields.

2.4 Comparison with Existing Systems

Commercial elevator systems use advanced sensors, redundancy hardware, and fault-detection circuits. While this project does not replicate industrial complexity, it reproduces the fundamental logic of elevator control systems, including direction-based decisions, task scheduling, and safety limits.

The primary difference lies in scale and hardware sophistication, not in operational concepts. The simplified design allows focus on system behavior rather than mechanical complexity, making it suitable for educational purposes while retaining realism.

2.5 Current Challenges and Project Significance

Modern elevator systems face challenges related to energy consumption, system optimization, and peak-hour congestion. Efficient scheduling algorithms are required to minimize wait time and unnecessary movement. This project introduces these challenges at an educational level through the implementation of a collective-selective control strategy.

The project is significant because it combines mechanical design, embedded programming, and system logic into a single working system. It helps students understand how software and hardware interact in real-time and strengthens problem-solving skills necessary for professional engineering practice.

Chapter 3

Execution Strategy

3.1 Project Strategy and Methods

The execution strategy of this project is based on a structured development approach that divides the system into independent subsystems: mechanical design, electronic integration, embedded programming, and communication interface implementation. This modular strategy ensures that each part can be individually developed and tested before full system integration.

Given the current academic level (7th semester, fourth-year engineering student), this project is considered fully feasible. The required knowledge base, including C programming, microcontroller operation, GPIO interfacing, timers, and serial communication, has already been acquired through coursework in embedded systems, electronics, and digital systems.

Mechanical fabrication using 3D printing is also within the author's capabilities, as the design involves basic structural components such as frames, mounts, and supports rather than precision-machined elements. The simplicity of continuous rotation servo motors further ensures feasibility while maintaining realistic behavior.

The scheduling algorithm implementation is simplified but representative of real elevator logic. The project applies a collectiveselective strategy which stores user requests, determines elevator direction, and executes floor servicing based on active calls. This method is selected because it is computationally efficient and appropriate for real-time microcontroller applications.

Advanced features such as optimization heuristics, motor feedback control, and fault diagnostics are identified as potential extensions that could be explored in future academic courses. These may include closed-loop motion control, enhanced scheduling algorithms, and networking capabilities for multi-device communication.

3.2 Project Implementation Plan

The project will be executed in six main phases to ensure systematic progress and system validation:

- Phase 1: System modeling and mechanical design using CAD software.
- Phase 2: Fabrication of parts using 3D printing and mechanical assembly.
- Phase 3: Hardware integration, including wiring of sensors, motors, and keypad.
- Phase 4: Embedded software development for GPIO control and motor actuation.
- Phase 5: Implementation of scheduling logic and UART communication.
- Phase 6: System testing, debugging, and performance evaluation.

The development is expected to be completed within one academic term. Each subsystem will be tested independently, followed by full system integration and verification.

3.3 Project Duration and Cost Justification

The duration of one academic term is adequate given the project's scale and available resources. Development does not require advanced manufacturing or specialized laboratory equipment, making the schedule realistic and achievable.

Estimated project costs are low due to the use of readily available components and academic hardware platforms. Major components include:

- STM32 NUCLEO-G474RE development board.
- Continuous-rotation servo motors.
- 4x4 matrix keypad.
- Magnetic sensors and permanent magnets.
- 3D printing materials.

The use of open-source development environments and HAL libraries eliminates software licensing costs. Additionally, the mechanical structure requires little material and no custom-machined parts, further reducing expenses.

Given the educational scope, low material cost, and realistic timeframe, the budget and development duration are appropriate and justified. The project focuses on engineering learning value rather than commercial production costs, making it well-suited for an academic environment.

Chapter 4

Project Performers Competencies

4.1 Technical Background

The project performer is currently a seventh-semester undergraduate student in engineering, possessing foundational and advanced knowledge in embedded systems and digital electronics. Coursework completed prior to this project includes subjects such as microcontrollers, digital systems, electronics, control systems, embedded programming, and system integration.

Through these courses, the performer has acquired experience in programming microcontrollers using the C language, configuring GPIO peripherals, managing timers, and implementing communication protocols such as UART. These competencies are directly applied in the development of the proposed elevator control system.

4.2 Hardware and Embedded System Skills

The performer has experience working with development boards based on ARM microcontrollers, including configuration and debugging using STM32CubeIDE. Skills include real-time hardware interfacing, motor control, keypad scanning, and sensor integration.

The project requires driving continuous rotation servo motors, detecting input from magnetic sensors, and reading user commands through a matrix keypad. The performer is able to implement these hardware functions through appropriate electronic interfacing and software control.

The ability to analyze system behavior using UART debugging and terminal-based monitoring further demonstrates readiness for integrating real-time feedback systems.

4.3 Mechanical Design and Fabrication

The project performer possesses practical skills in mechanical design using CAD software for the development of custom components such as elevator frames and cabin enclosures. Experience with 3D printing allows for rapid prototyping and testing of mechanical structures.

Basic understanding of mechanical constraints, load distribution, tolerances, and assembly processes ensures that the mechanical design is implementable and stable enough for testing and demonstration.

4.4 Software Design and Problem Solving

The development of scheduling logic and system control algorithms requires structured thinking and debugging skills. The performer is experienced in designing logical state machines, implementing event-based logic, and managing multiple system variables in real time.

The performer also demonstrates the ability to handle software debugging, reading serial logs, and verifying program behavior through systematic testing. These skills are essential for executing a real-time control project involving multiple actuators and sensors.

4.5 Capability for Project Completion

Based on current academic training, previous laboratory experience, and independent technical practice, the project performer is capable of completing all major components of the proposed project.

The project directly aligns with previously completed coursework and requires no advanced tools beyond those already accessible. Any required additional knowledge, such as optimization of scheduling or advanced peripheral configurations, can be acquired through documentation, experimentation, and supplementary coursework.

The combination of academic training, hands-on experience, and self-directed learning ability provides sufficient foundation for successful completion of this project.

Chapter 5

Utilization Plan and Expected Outcomes

5.1 Project Utilization Plan

The developed elevator automation system is intended primarily as an educational and experimental platform for embedded systems research and instruction. The completed system will be used to demonstrate real-time control, scheduling algorithms, and sensor integration in a physical multi-actuator system.

The project may be utilized in laboratory environments for teaching topics such as microcontroller based control, input/output interfacing, scheduling strategies, and serial communication. By modifying the software, the system can also serve as a research testbed for experimenting with alternative scheduling algorithms, control approaches, and communication methods.

In addition to academic use, the system may be adapted as a prototype model for automation demonstrations in exhibitions or presentations. The modular design allows individual components, such as the keypad interface, motor control module, or sensor subsystem, to be reused in other automation projects involving robotics or industrial control.

The UART monitoring system may also be used as a debugging and visualization tool, allowing users to observe system behavior and understand the internal logic in real time. This improves usability for educational settings and assists in performance analysis.

5.2 Expected Outcomes and Impacts

5.2.1 Scientific and Technological Impact

From a technical standpoint, the project will result in a functioning embedded control system integrating sensing, actuation, communication, and scheduling logic. The final implementation demonstrates how real-time systems operate under resource constraints and showcases standard automation techniques used in industrial control systems.

The system provides a foundation for further research into real-time optimization, fault detection, and autonomous decision-making in embedded environments.

5.2.2 Economic and Industrial Perspective

Although the project is not intended as a commercial product, it demonstrates the fundamental architecture used in transportation automation systems. Knowledge gained through the development process may be transferred to industrial applications such as material handling, warehouse automation, and manufacturing systems.

The low material cost and minimal infrastructure required make this project suitable for educational institutions seeking affordable automation platforms.

5.2.3 Social and Educational Impact

The primary impact of this project is educational. By constructing and programming a real automation system, students develop technical competence, problem-solving skills, and system-level thinking. The project also promotes interdisciplinary learning by combining mechanical design, electronics, and software development into a single system.

Understanding automation systems also fosters awareness of how technology improves safety and efficiency in daily life.

5.2.4 Differentiation from Existing Solutions

Unlike commercial elevator systems which prioritize reliability and redundancy, this project emphasizes transparency and educational clarity. Internal system states are displayed through UART communication, allowing users to observe decision-making processes in real time.

The system is lightweight, modifiable, and open for experimentation, making it distinct from industrial platforms that are typically closed and proprietary. This adaptability enables users to explore "what-if" scenarios that would not be possible in commercial installations.

Chapter 6

System Framework

The proposed system is a fully functional dual-elevator model operating within a five-floor environment. The framework integrates mechanical motion, sensor feedback, user interaction, and embedded decision-making, forming a complete miniature elevator automation system. The matrix keypad is used as the primary user interface for selecting both origin and destination floors.

6.1 Overall System Architecture

The system consists of two independent elevators housed within parallel vertical structures. Each elevator cabin is suspended by a rope mechanism driven by a continuous-rotation servo motor connected to a drum. The motion of each elevator is monitored using magnetic limit switches placed at the top and bottom of the shaft. A magnet attached to each cabin triggers these sensors to prevent overtravel and ensure safe operation.

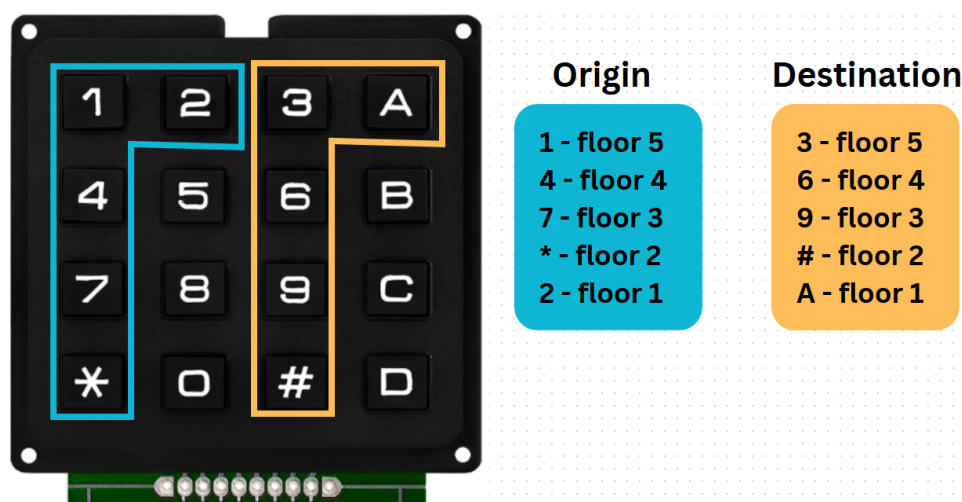


Figure 6.1: Input system

A 44 matrix keypad serves as the main input interface. The keypad is divided into two logical zones:

- **Origin Selection Zone:** Keys 1, 4, 7, *, 2 correspond to floors 5, 4, 3, 2, and 1 respectively.
- **Destination Selection Zone:** Keys 3, 6, 9, #, A correspond to floors 5, 4, 3, 2, and 1 respectively.

This mapping allows the user to define both the calling floor and the desired destination, similar to a real elevator environment.

6.2 System Workflow

The complete operational flow of the system proceeds as follows:

1. **User Input Acquisition:** The user selects an origin floor and then a destination floor using the keypad. The STM32 microcontroller scans the keypad matrix to detect the pressed keys and registers the request.
2. **Door-Closing Simulation Delay:** After receiving a valid request, the system introduces a short delay to mimic door-closing time before elevator movement begins. The request is recorded and broadcast through UART to the PC terminal.
3. **Scheduling and Elevator Assignment:** A collective-selective scheduling algorithm determines which of the two elevators should respond based on:
 - current elevator positions,
 - direction of travel,
 - pending requests,
 - proximity to the origin floor.

The elevator that minimizes travel time and maximizes efficiency is selected.

4. **Motion Control:** The assigned elevator begins moving toward the origin floor. The servo motor rotates the drum mechanism to raise or lower the cabin. Real-time data such as elevator position, status, and active requests is transmitted continuously over UART to the PC terminal.
5. **Floor Verification and Limit Protection:** Each elevator includes:
 - one magnetic sensor at the bottom,
 - one magnetic sensor at the top,
 - one magnet attached to the elevator cabin.

As the cabin travels, the magnet triggers the sensors when reaching the top or bottom limits. The system immediately stops motor movement upon detection, preventing mechanical overtravel or structural stress.

6. **Destination Handling:** After reaching the origin floor, the system simulates a door-opening period, then proceeds to the selected destination floor following the same safety and scheduling logic.
7. **Completion and Idle State:** Once the request is complete, the elevator either enters an idle state or continues processing remaining requests in the queue. All events are logged through UART for monitoring and debugging.

6.3 System Monitoring via UART

A key component of the system framework is real-time system observability. The microcontroller transmits structured data to the PC terminal, including:

- elevator positions and directions,
- active floor requests,
- motor activation states,
- sensor limit detections,
- scheduling decisions.

This feature enables continuous monitoring, simplifies debugging, and enhances transparency of system behavior.

6.4 Summary of System Behavior

Overall, the system behaves as a compact yet realistic elevator automation model. It accepts user inputs, assigns elevators intelligently through a scheduling algorithm, ensures safe movement with magnetic limit sensors, controls motion through servo-driven mechanics, and reports all internal states through UART communication. The system effectively demonstrates key concepts of embedded control, safety interlocks, multi-elevator coordination, and real-time communication.

Chapter 7

Inputs and Outputs

7.1 Input Section Specification

7.1.1 Purpose of Input

The input subsystem collects all user commands and system feedback necessary for elevator operation. User inputs from the 4x4 matrix keypad define the origin floor and destination floor, while magnetic limit sensors detect physical boundaries and ensure safe operation.

This information is passed to the elevator control logic (Rules), which determines how the elevators should respond.

7.1.2 Rules (Input Interpretation and Flow)

The rules governing input interpretation are as follows:

- Each keypad press is scanned using row-column detection and mapped to a corresponding floor request.
- Inputs are categorized into two sets:
 - **Origin requests** (keys: 1, 4, 7, *, 2)
 - **Destination requests** (keys: 3, 6, 9, #, A)
- Once a valid origin-destination pair is entered, the request is added to a centralized scheduling queue.
- Hall sensors provide immediate hard-limit detection; if triggered during input processing, any pending movement is temporarily blocked.
- All validated input data is forwarded to the scheduling logic, which determines elevator allocation and pathing.

7.1.3 Required Components

- 4x4 Matrix Keypad
- Four Digital Hall Sensors
- Magnet pairs (top and bottom of shafts)
- STM32 NUCLEO-G474RE

7.1.4 Input Circuit

- **Keypad:** Rows connected to GPIO outputs; columns connected to GPIO inputs with pull-ups. Keypresses create rowcolumn closures detected by the MCU.
- **Hall Sensors:** Digital output sensors (opencollector or pushpull depending on model). Output goes LOW when a magnet is detected.

7.1.5 Description of Input Components

Input 1: 4x4 Matrix Keypad

- **Purpose:** User selects origin and destination floors.
- **Mapping:** Origin 1, 4, 7, *, 2 Destination 3, 6, 9, #, A
- **Principle:** Row-column scanning.

Input 2: Hall Effect Sensors

- **Purpose:** Detect top and bottom shaft limits.
- **Principle:** Digital switching using magnetic presence.

7.1.6 Input Processing Method

1. MCU scans keypad rows and monitors hall sensor states.
2. Origin and destination keys are interpreted and stored.
3. Requests are validated and added to a scheduling queue.
4. Magnetic-limit states are checked to enforce safe boundaries.
5. The processed inputs are sent into the scheduling and elevator allocation logic.

7.2 Output Section Specification

7.2.1 Purpose of Output

The output subsystem generates all physical elevator actions and system feedback. Servo motors execute movement according to scheduling decisions, while UART communication transmits real-time system states to a PC terminal.

Outputs reflect the results of the Rules block, transforming decisions into motion and visible information.

7.2.2 Rules (Output Decision Logic)

The Rules governing output behavior are:

- The scheduling algorithm selects which elevator will service each request based on:
 - elevator proximity to origin floor
 - current elevator direction
 - queue load
- Output PWM signals are generated to:
 - move upward (PWM > 1.5 ms)
 - move downward (PWM < 1.5 ms)
 - stop (PWM = 1.5 ms)
- Movement stops immediately if a magnetic limit sensor is triggered.
- UART continuously transmits:
 - elevator positions
 - movement direction
 - active requests
 - safety events
 - scheduling decisions

7.2.3 Required Components

- Two continuous-rotation servo motors
- Power supply (5V)
- STM32 PWM outputs
- UART communication link

7.2.4 Output Circuit

- **Servo Motors:** Driven via PWM pins connected to STM32 timers. External 5V supply powers motors; ground is shared with MCU.
- **UART Terminal:** TX pin from STM32 sends formatted data frames through the ST-Link USB interface to a PC.

7.2.5 Description of Output Components

Output 1: Servo Motors

- **Purpose:** Generate vertical motion.
- **Principle:** PWM width determines rotational direction and speed.

Output 2: UART Serial Output

- **Purpose:** Provide live system monitoring.
- **Data Sent:** Floor values, motor states, scheduler events, limit triggers.

7.2.6 Output Processing Method

1. Scheduler assigns an elevator to each request.
2. PWM output begins motion in the appropriate direction.
3. System monitors the travel time to determine when a floor has been reached.
4. Servo stops upon arrival or when a safety sensor is triggered.
5. UART continuously outputs system information for debugging and demonstration.

Chapter 8

Phase 1; Mechanical Design

The mechanical components of the dualelevator system were designed using SolidWorks, following an iterative CAD process that ensured correct fitting, structural stability, and compatibility with the continuousrotation servos employed in the project. All the models were later fabricated using FDM 3D printing, allowing rapid prototyping and precise control over tolerances. The final assembly consists of two elevator boxes (one for each elevator) and a unified supporting structure that houses and guides their motion.

8.1 Elevator Box Design

Each elevator car was modelled as a rectangular enclosure with an internal hollow volume, as illustrated in Figure 8.1. The goal of the box was to provide a rigid platform for the servodriven motion while maintaining a lightweight geometry suitable for the torque output of micro servos.

The final external dimensions of each box are:

- Height: **29.4 mm**
- Depth: **29.4 mm**
- Length: **49.4 mm**

Since the system incorporates two independent elevators, two identical boxes were manufactured. The design includes reinforced top and bottom surfaces and a central alignment hole to simplify connection with the vertical shaft.

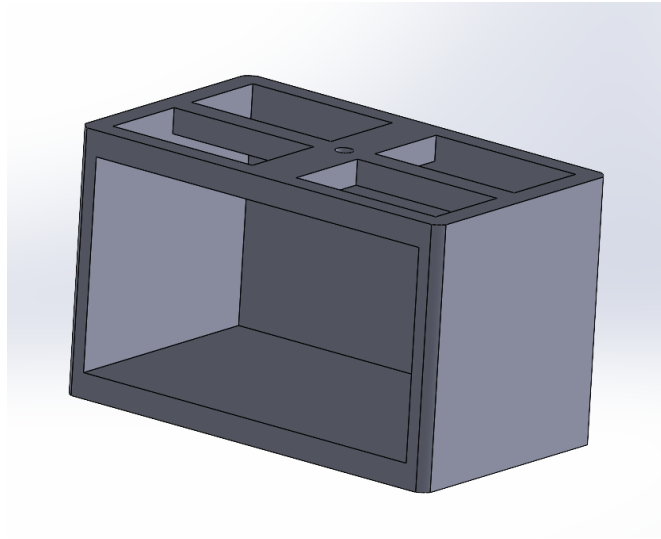


Figure 8.1: Elevator box model designed in SolidWorks .

8.2 Supporting Structure

The main elevator shaft was designed as a tall, lightweight truss structure composed of repeated triangular patterns to improve stiffness without significantly increasing print time or material consumption. Figure 8.2 shows an isometric view of the assembled structure.

To ensure proper fit, the internal compartment of the structure was dimensioned to:

- Length: **50 mm**
- Height: **30 mm**
- Depth: **30 mm**

Because 3D printers introduce dimensional inaccuracies, especially along the XY plane, a tolerance of **6 mm** was subtracted from the box's nominal dimensions. This ensured smooth vertical motion while preventing excessive lateral play. In practice, the box fits the shaft with adequate clearance, even after accounting for wall expansion caused by extrusion width variations.

The structural walls of the elevator shaft were printed with a thickness of **3 mm** throughout most of the geometry. Only the base section was increased to **6 mm** to reinforce the area that supports the entire vertical load and the mechanical torque from the servo. This base plate also includes lateral stabilizers to prevent the structure from tipping during operation.

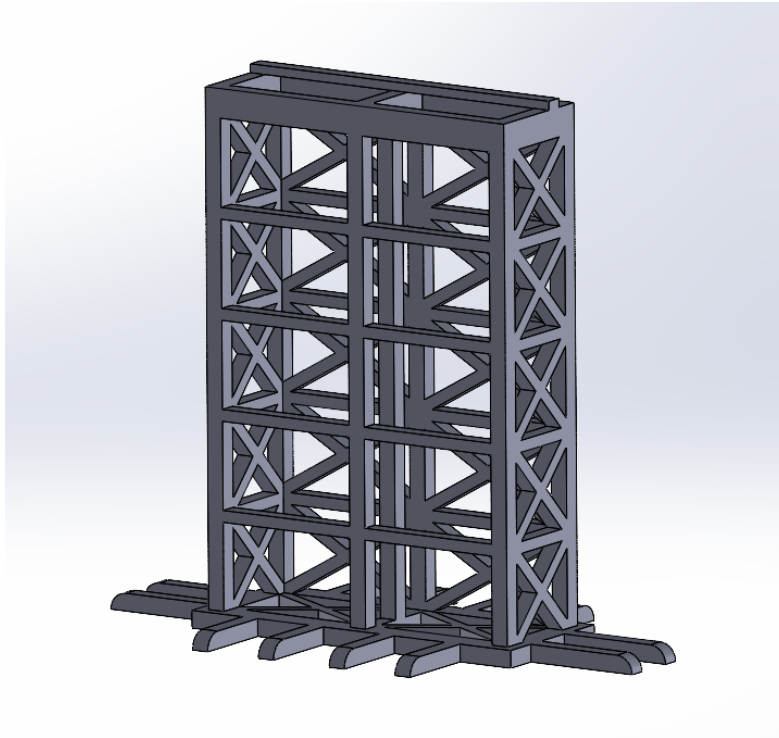


Figure 8.2: Isometric view of the complete elevator shaft, showing the truss reinforcement pattern.

8.3 Base and Rail System

Figure 8.3 shows the top view of the lower section of the structure, where multiple raillike extensions were incorporated. These elements improve contact area with the ground and increase stability. The crossing pattern inside the rectangular frame provides torsional rigidity and maintains alignment between both elevator lanes.

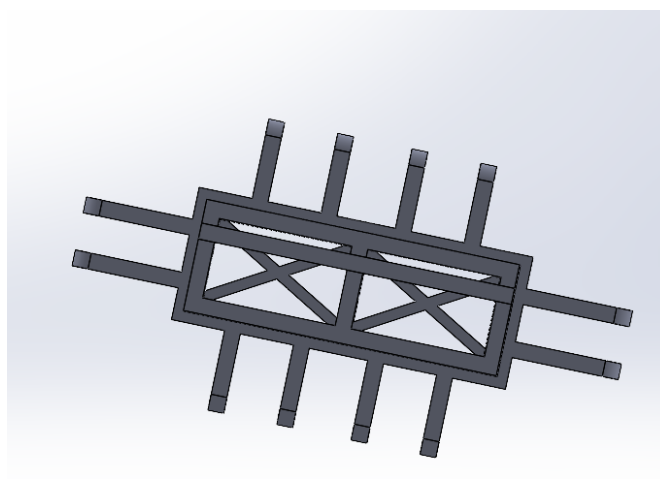


Figure 8.3: Top view of the base structure with integrated reinforcement elements.

8.4 Final Assembly

Once the individual parts were printed, the two elevator boxes were inserted into their respective lanes in the shaft. The tolerances behaved as expected, and both elevators were able to slide freely along the guide without friction or mechanical interference. The final assembly is shown in Figure 8.4, which represents the complete mechanical subsystem prior to integration with electronics and control.

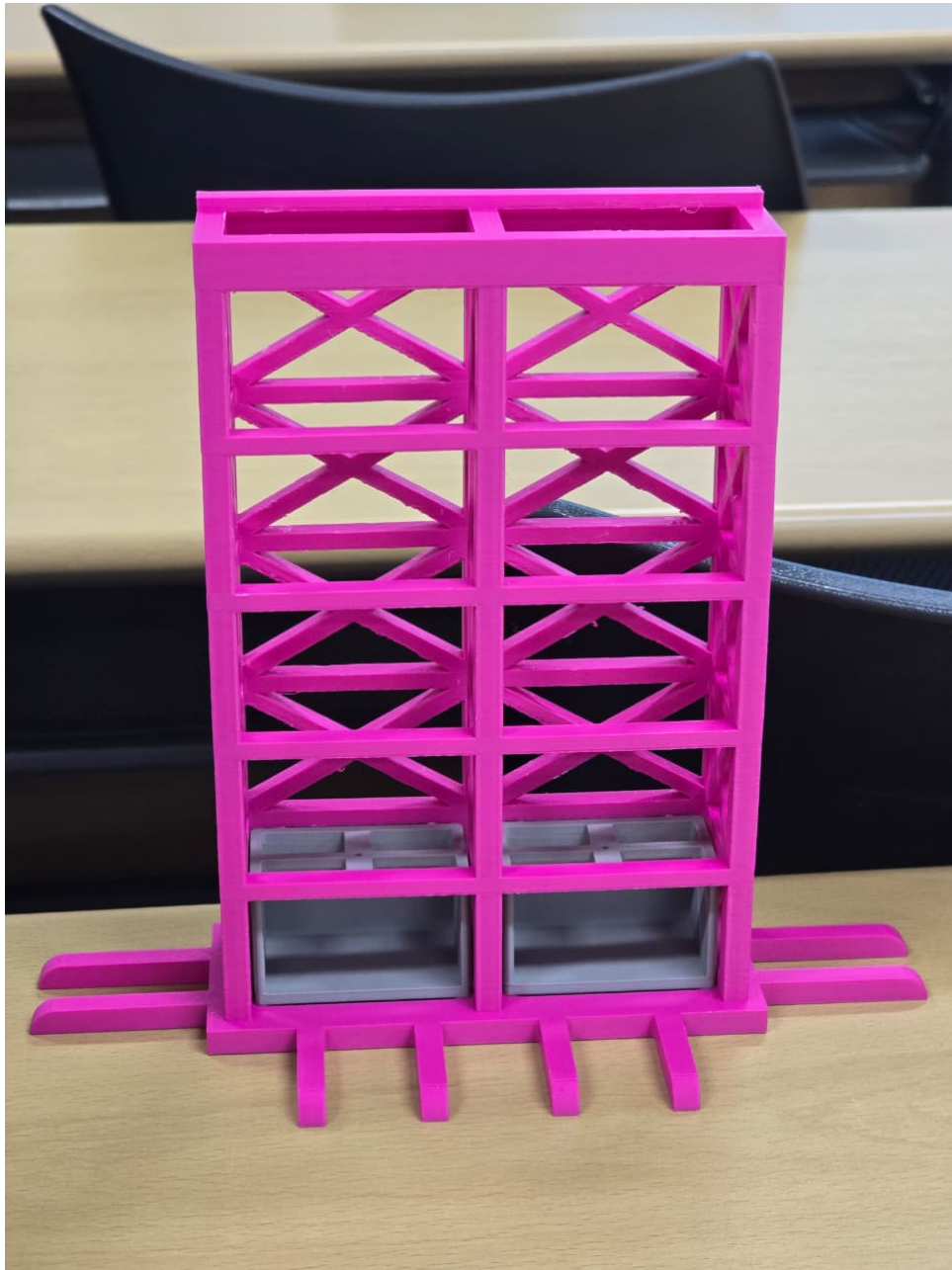


Figure 8.4: Final assembly of the mechanical elevator structure.

Overall, the mechanical design successfully met the constraints of low weight, structural stability, ease of printing, and compatibility with the servodriven motion control system implemented in later phases of the project.

Chapter 9

Phase 2; Hardware Integration

This phase focuses on the integration of all electrical and electronic components required for the correct operation of the elevator system. The objective of this stage was to ensure reliable communication between the control unit, user input interface, and actuators, while maintaining electrical safety and signal integrity.

The complete system was assembled on a prototyping breadboard to allow flexibility during testing and debugging. Figure references throughout this chapter illustrate the final wiring layout and pin assignments used during implementation.

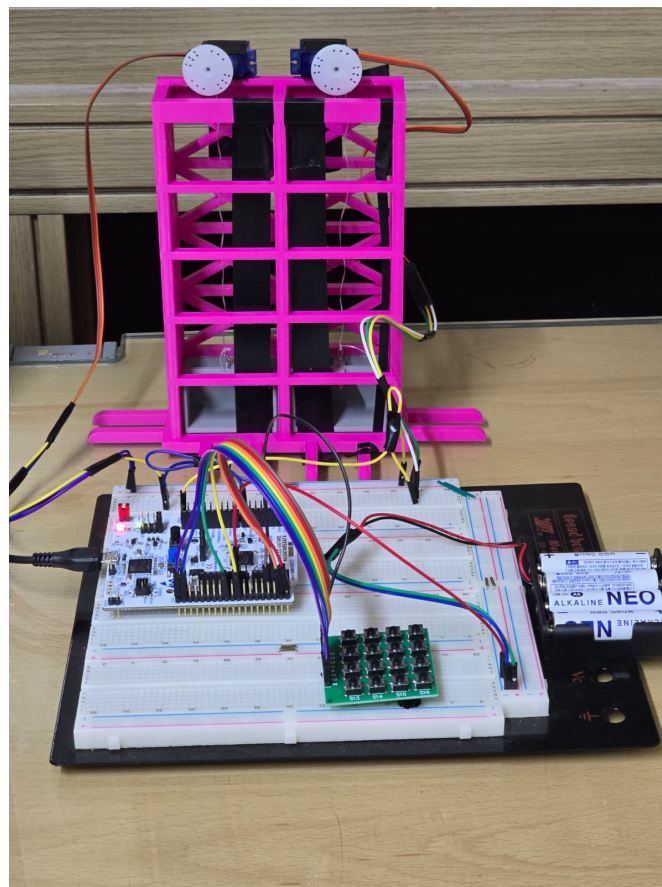


Figure 9.1: Electrical connections.

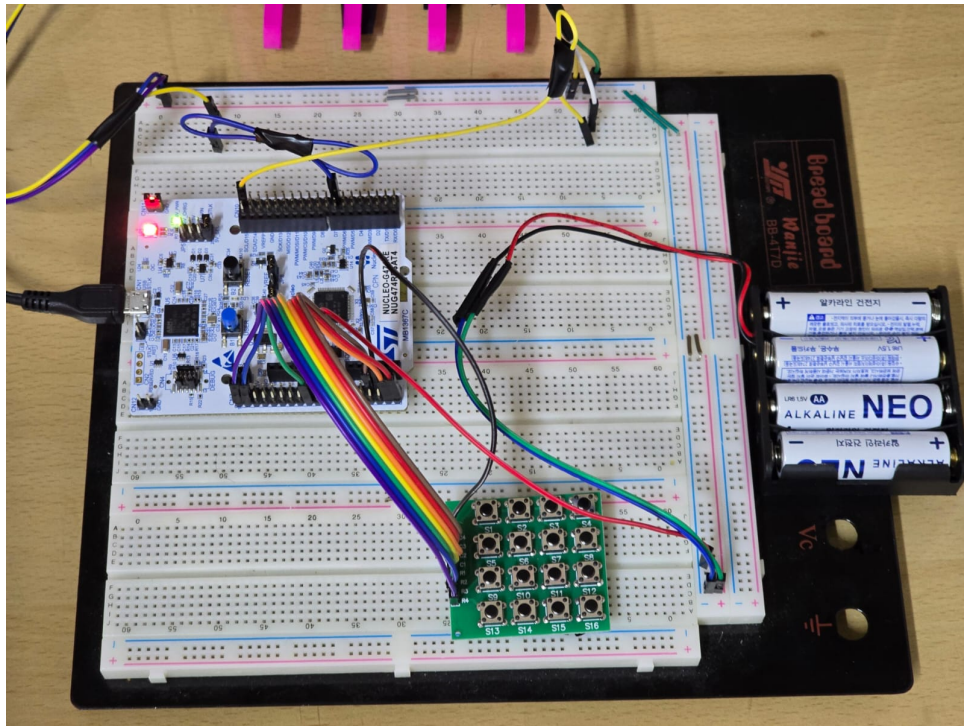


Figure 9.2: Electrical connections

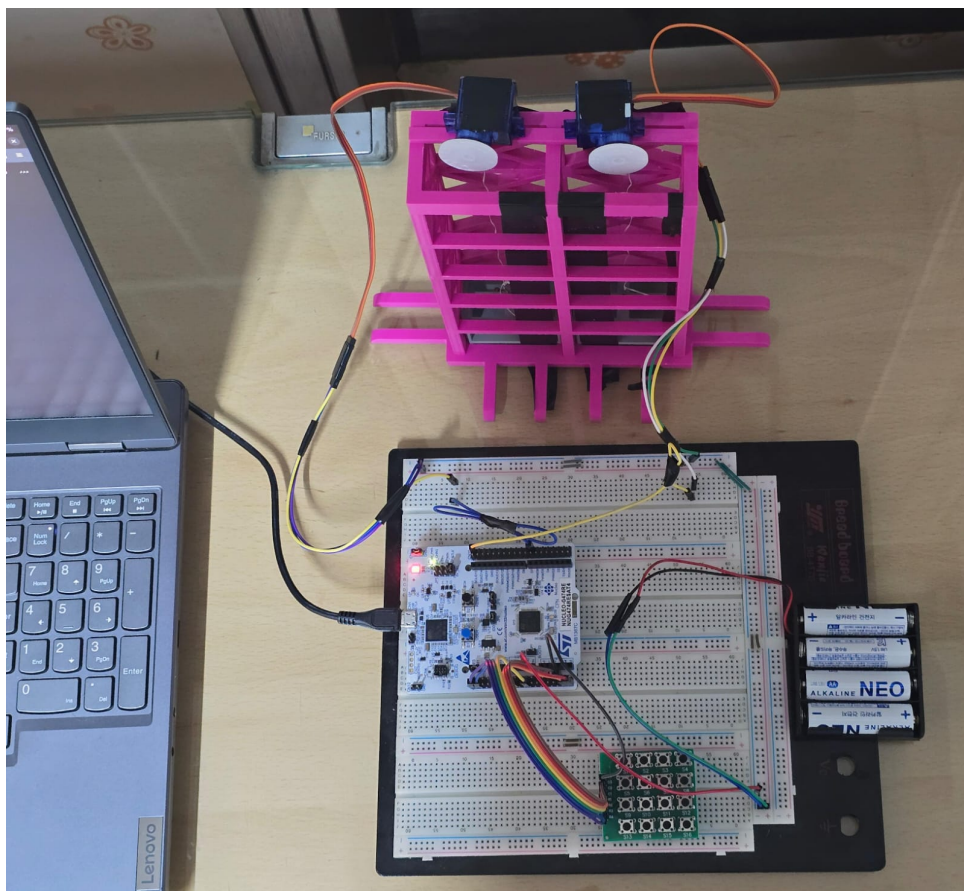


Figure 9.3: Electrical connections

9.1 System Overview

The hardware system is composed of the following main elements:

- An **STM32 NUCLEO-G474RE** development board, used as the main controller.
- Two **FS90R continuous-rotation servomotors**, responsible for the vertical motion of each elevator cabin.
- A **4×4 matrix keypad**, used as the user input interface for selecting origin and destination floors.
- An external **battery supply**, dedicated to powering the servomotors.
- A breadboard and jumper wires for signal routing and power distribution.

All components were interconnected following good embedded systems practices, such as common ground reference, separation of logic and motor power, and correct GPIO configuration.

9.2 Microcontroller and Power Distribution

The STM32 NUCLEO-G474RE board was powered through its USB connector, which also enabled programming and UART-based debugging. The servomotors were powered using an external battery pack to avoid overloading the onboard voltage regulator.

A common ground was shared between the STM32 board and the external power supply. This step was critical to ensure stable PWM signal interpretation by the servomotors.

9.3 Servo Motor Connections

Each elevator is driven by one FS90R continuous-rotation servo motor. These motors require a PWM control signal with a period of 20 ms (50 Hz), where the pulse width determines direction and speed.

The servomotor connections were implemented as follows:

- **Elevator 1 servo:**
 - PWM signal connected to PB1 (TIM3_CH4)
- **Elevator 2 servo:**
 - PWM signal connected to PC8 (TIM20_CH3)
- Both servos share a common ground with the microcontroller.
- Power is supplied externally to provide sufficient current during operation.

The physical placement of the servos on top of the elevator structure can be observed in the system photographs, where each servo drives its corresponding elevator cabin through a pulley and cable mechanism.

9.4 Matrix Keypad Interface

User input is handled through a 4×4 matrix keypad. The keypad was interfaced using eight GPIO pins configured as four row outputs and four column inputs with internal pull-up resistors.

The keypad wiring follows a standard matrix scanning configuration:

- Rows are configured as digital outputs.
- Columns are configured as digital inputs with pull-up resistors.
- A key press is detected by pulling a column line low when its corresponding row is driven low.

This configuration minimizes the number of GPIO pins required while allowing reliable detection of all key combinations.

9.5 CubeIDE Pin Configuration

The GPIO and peripheral configuration was performed using STM32CubeIDE. The pinout was carefully selected to match the physical wiring and hardware constraints of the system.

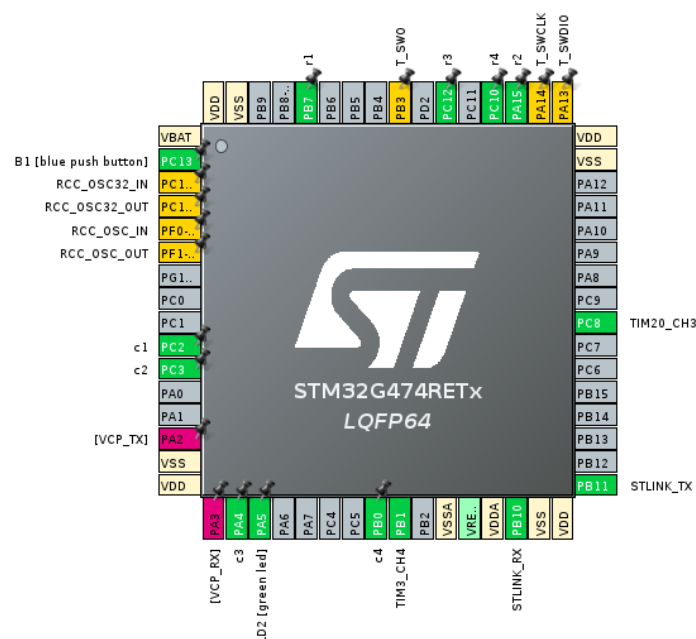


Figure 9.4: CubeIDE Pin Configuration

Key pin assignments include:

- PB1: PWM output for Elevator 1 (TIM3_CH4)
- PC8: PWM output for Elevator 2 (TIM20_CH3)
- Multiple GPIO pins on ports A, B, and C: keypad row and column connections
- UART pins: used for debugging and system monitoring through a serial terminal

A visual representation of the STM32CubeIDE pinout configuration is shown in the corresponding figure, highlighting the timers, GPIO assignments, and peripheral usage.

9.6 Physical Assembly

Figures included in this chapter show the complete physical assembly of the system, including:

- The STM32 Nucleo board mounted on a breadboard
- The matrix keypad connected via ribbon cables
- The external battery pack supplying the servomotors
- The elevator structure with both servos mounted on top

This assembly allowed direct observation of both mechanical motion and electrical behavior during testing.

9.7 Demonstration Video

To complement the static images and wiring diagrams, a video demonstration of the electrical connections and system operation is provided below.

- **Video:** <https://youtu.be/ulnk5hJibjI?si=Kf-K0Fx7CQR4lFax>

This video shows the complete setup, including power distribution, keypad interaction, and elevator motion in response to user inputs.

Chapter 10

Phase 3; Code and Implementation of Scheduling Logic

This phase focuses on the complete software implementation of the elevator system. The main objective of the code is to: (1) acquire user requests from the keypad, (2) store and manage those requests, (3) assign each request to the most convenient elevator, and (4) execute the elevator motion and door timing using a deterministic state machine. The implementation was written in C using STM32 HAL libraries inside STM32CubeIDE.

The final system supports two independent elevators, each controlled by a dedicated hardware timer channel generating a PWM signal for a continuous-rotation servo motor. Additionally, the implementation introduces per-servo calibration parameters in order to compensate for manufacturing tolerances (neutral drift, slightly different speed response, and PWM jitter).

The complete source code for this project can be access at:

<https://github.com/raymundo140/stm32-dual-elevator-embedded-system>

10.1 Software Architecture Overview

At a high level, the firmware is organized into four main modules, all inside `main.c`:

- **Input layer (Keypad scanning):** reads the matrix keypad and returns the corresponding character.
- **Actuation layer (Servo control):** converts “UP / DOWN / STOP” actions into PWM microseconds and writes them into the proper timer channel.
- **Scheduling layer (Request queue + elevator selection):** stores requests and assigns them to elevators using a scoring function.
- **Execution layer (State machine):** updates each elevator over time using a moving/-doors logic driven by `HAL_GetTick()`.

This separation keeps the implementation modular. The keypad never directly moves motors; instead, it generates requests. The scheduler never touches GPIO; it only decides which elevator should take a job. The state machine is the only part of the code that decides when to move, stop, or open/close doors.

10.2 Scheduling Algorithm Used

The system implements a **simple greedy scheduling algorithm with a stable scoring function**. Each time there is at least one idle elevator and at least one pending request, the scheduler evaluates all elevators and selects the one with the lowest score.

The score considers:

- **Distance to pickup floor:** elevators closer to the request origin are preferred.
- **Idle preference:** idle elevators receive a much lower score than busy ones.
- **Direction similarity (minor bonus):** if an elevator is already moving in the same direction as the request, the score is slightly reduced (only used as a tie-break / small bias).

This approach is intentionally simple and deterministic. It does not attempt to globally optimize all future requests (which would require more complex logic). Instead, it makes a locally optimal assignment each time an elevator becomes available, which is appropriate for the small-scale system and the lab objectives.

10.3 Code Explanation

This section explains the full code, including all definitions, data structures, functions, and the main loop. Code fragments are included using `tcolorbox` blocks in the required format.

10.4 Includes and Definitions

The program starts by including the HAL-generated header and the additional project headers.

Includes and Dependencies

```
#include "main.h"

/* USER CODE BEGIN Includes */
#include "myprintf.h"
#include "matrix_keypad.h"
#include <stdlib.h> // abs()
```

```
/* USER CODE END Includes */
```

`main.h` contains the MCU configuration, pin definitions, and HAL declarations. The file `myprintf.h` enables printing debug messages through UART. Finally, `stdlib.h` is included to use `abs()`, which is required by the scheduling score function.

10.4.1 Keypad GPIO Definitions and Timing Constants

The keypad is connected as a 4×4 matrix, where rows are outputs and columns are inputs with pull-up resistors. The timing constants define the simplified elevator behavior model.

Keypad Pins and Timing Parameters

```
// ===== KEYPAD CONNECTIONS =====  
  
#define R1_PORT GPIOB  
#define R1_PIN  GPIO_PIN_7  
#define R2_PORT GPIOA  
#define R2_PIN  GPIO_PIN_15  
#define R3_PORT GPIOC  
#define R3_PIN  GPIO_PIN_12  
#define R4_PORT GPIOC  
#define R4_PIN  GPIO_PIN_10  
  
#define C1_PORT GPIOC  
#define C1_PIN  GPIO_PIN_2  
#define C2_PORT GPIOC  
#define C2_PIN  GPIO_PIN_3  
#define C3_PORT GPIOA  
#define C3_PIN  GPIO_PIN_4  
#define C4_PORT GPIOB  
#define C4_PIN  GPIO_PIN_0  
  
// Timing model  
#define TIME_PER_FLOOR_MS 1500  
#define DOOR_TIME_MS      2000
```

`TIME_PER_FLOOR_MS` models the time required for an elevator to move one floor, and `DOOR_TIME_MS` models the time required to keep doors open before continuing.

10.5 Keypad Scanning Logic

The keypad scanning function drives one row low at a time and checks which column becomes low, indicating a pressed key. The function returns a character representing the pressed key, or 0 if no key is pressed.

Matrix Keypad Scan Function

```
static char read_keypad(void)
{
    HAL_GPIO_WritePin(R1_PORT, R1_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(R2_PORT, R2_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(R3_PORT, R3_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(R4_PORT, R4_PIN, GPIO_PIN_SET);

    #define CHECK_COL(port, pin, value) \
        if (HAL_GPIO_ReadPin((port), (pin)) == GPIO_PIN_RESET) { \
            while (HAL_GPIO_ReadPin((port), (pin)) == GPIO_PIN_RESET); \
            HAL_Delay(10); \
            return (value); \
        }

    HAL_GPIO_WritePin(R1_PORT, R1_PIN, GPIO_PIN_RESET);
    HAL_Delay(1);
    CHECK_COL(C1_PORT, C1_PIN, '1');
    CHECK_COL(C2_PORT, C2_PIN, '2');
    CHECK_COL(C3_PORT, C3_PIN, '3');
    CHECK_COL(C4_PORT, C4_PIN, 'A');
    HAL_GPIO_WritePin(R1_PORT, R1_PIN, GPIO_PIN_SET);

    HAL_GPIO_WritePin(R2_PORT, R2_PIN, GPIO_PIN_RESET);
    HAL_Delay(1);
    CHECK_COL(C1_PORT, C1_PIN, '4');
    CHECK_COL(C2_PORT, C2_PIN, '5');
    CHECK_COL(C3_PORT, C3_PIN, '6');
    CHECK_COL(C4_PORT, C4_PIN, 'B');
    HAL_GPIO_WritePin(R2_PORT, R2_PIN, GPIO_PIN_SET);

    HAL_GPIO_WritePin(R3_PORT, R3_PIN, GPIO_PIN_RESET);
    HAL_Delay(1);
    CHECK_COL(C1_PORT, C1_PIN, '7');
    CHECK_COL(C2_PORT, C2_PIN, '8');
    CHECK_COL(C3_PORT, C3_PIN, '9');
```

```
CHECK_COL(C4_PORT, C4_PIN, 'C');
HAL_GPIO_WritePin(R3_PORT, R3_PIN, GPIO_PIN_SET);

HAL_GPIO_WritePin(R4_PORT, R4_PIN, GPIO_PIN_RESET);
HAL_Delay(1);
CHECK_COL(C1_PORT, C1_PIN, '*');
CHECK_COL(C2_PORT, C2_PIN, '0');
CHECK_COL(C3_PORT, C3_PIN, '#');
CHECK_COL(C4_PORT, C4_PIN, 'D');
HAL_GPIO_WritePin(R4_PORT, R4_PIN, GPIO_PIN_SET);

return 0;
}
```

The macro `CHECK_COL` checks each column line. If a pressed key is detected (column reads low), the function waits until the key is released (simple software debouncing), delays 10 ms for stability, and returns the corresponding character.

10.6 Servo Motor Control and Per-Servo Calibration

10.6.1 Low-Level PWM Write Function

Servos are driven by PWM pulses expressed in microseconds. A helper function clamps the pulse to safe limits and writes the value into the capture/compare register.

Low-Level Servo PWM Function

```
static void Servo_SetPulse_us(TIM_HandleTypeDef *htim,
    uint32_t channel, uint16_t us)
{
    if (us < 700) us = 700;
    if (us > 2300) us = 2300;
    __HAL_TIM_SET_COMPARE(htim, channel, us);
}
```

The timers were configured so that 1 timer tick corresponds to 1 μ s. Therefore, writing 1500 produces a 1500 μ s pulse.

10.6.2 Motivation for Per-Servo Profiles

Continuous-rotation servos do not always stop perfectly at exactly $1500\mu\text{s}$. Manufacturing tolerances and internal potentiometer offsets can cause “creep” where the servo moves slightly even when commanded to stop. To solve this, a **per-servo profile** is defined, allowing each servo to have its own neutral point and its own speed commands.

Servo Profile Structure and Configuration

```
typedef struct {
    uint16_t neutral_us;    // true stop for that servo
    uint16_t up_us;         // pulse to go UP
    uint16_t down_us;       // pulse to go DOWN
    uint16_t deadband_us;   // +/- around neutral treated as neutral
} ServoProfile_t;

static ServoProfile_t servo_cfg[3] = {
    {0,0,0,0}, // index 0 unused
    // Elevator 1 (TIM3 CH4)
    { .neutral_us = 1500, .up_us = 1562, .down_us = 1413, .deadband_us = 6 },
    // Elevator 2 (TIM20 CH3)
    { .neutral_us = 1495, .up_us = 1565, .down_us = 1390, .deadband_us = 6 }
};
```

The fields represent:

- **neutral_us**: the pulse width where that specific servo stops.
- **up_us and down_us**: pulse widths that define the motion speeds.
- **deadband_us**: a safety region around neutral where small variations are treated as neutral.

10.6.3 Deadband Filtering

PWM jitter or small variations around the neutral pulse can cause undesired servo motion. To prevent this, a deadband is implemented.

Deadband Application Function

```
static uint16_t servo_apply_deadband(uint8_t elev_id, uint16_t cmd_us)
{
    uint16_t n = servo_cfg[elev_id].neutral_us;
    uint16_t db = servo_cfg[elev_id].deadband_us;
```

```
int diff = (int)cmd_us - (int)n;
if (diff > -(int)db && diff < (int)db) return n;
return cmd_us;
}
```

If the command pulse is within $\pm$ deadband of the neutral point, the function forces the command to exactly neutral. This reduces unintended drift.

10.6.4 Elevator PWM Mapping and High-Level Commands

Each elevator servo is connected to a different timer/channel, so a mapping function routes the PWM command to the correct output. Higher-level functions (`stop`, `up`, `down`) abstract the servo profile usage.

PWM Routing and Motion Commands

```
static void elevator_pwm_raw(uint8_t elev_id, uint16_t us)
{
    if (elev_id == 1) {
        Servo_SetPulse_us(&htim3, TIM_CHANNEL_4, us);    // PB1
    } else {
        Servo_SetPulse_us(&htim20, TIM_CHANNEL_3, us);    // PC8
    }
}

static void elevator_stop(uint8_t elev_id)
{
    uint16_t cmd = servo_cfg[elev_id].neutral_us;
    elevator_pwm_raw(elev_id, cmd);
}

static void elevator_up(uint8_t elev_id)
{
    uint16_t cmd = servo_apply_deadband(elev_id, servo_cfg[elev_id].up_us);
    elevator_pwm_raw(elev_id, cmd);
}

static void elevator_down(uint8_t elev_id)
{
    uint16_t cmd = servo_apply_deadband(elev_id, servo_cfg[elev_id].down_us);
    elevator_pwm_raw(elev_id, cmd);
}
```

This approach ensures each servo can be tuned independently while keeping the rest of the algorithm servo-agnostic.

10.7 Data Structures for Requests and Elevator State

The system is modeled with:

- A **request queue** storing origin and destination pairs.
- Two **elevator objects**, each with its own state machine and job phase.

Elevator and Request Data Structures

```
#define NUM_FLOORS      5
#define MAX_REQUESTS   10
#define NUM_ELEVATORS   2

typedef enum { DIR_IDLE=0, DIR_UP, DIR_DOWN } Direction_t;

typedef enum {
    ELEV_IDLE = 0,
    ELEV_MOVING,
    ELEV_DOORS
} ElevatorState_t;

typedef enum {
    PHASE_TO_PICKUP = 0,
    PHASE_TO_DROPOFF
} JobPhase_t;

typedef struct {
    uint8_t origin;
    uint8_t destination;
    Direction_t dir;
} Request_t;

typedef struct {
    uint8_t id;
    uint8_t current_floor;

    uint8_t pickup_floor;
    uint8_t dropoff_floor;
    JobPhase_t phase;
```

```
    Direction_t dir;
    ElevatorState_t state;
    uint32_t state_start_time;

    uint8_t busy;
} Elevator_t;
```

Key design details:

- **state_start_time** stores the timestamp of the last state transition (movement step or door opening).
- **phase** separates the job into two steps: go to pickup, then go to dropoff.
- **busy** indicates whether the elevator is currently assigned to a job.

Two elevator instances are declared and initialized to floor 1.

Initial Elevator States

```
static Elevator_t elevators[NUM_ELEVATORS] = {
    { .id=1, .current_floor=1, .busy=0, .state=ELEV_IDLE, .dir=DIR_IDLE,
      .phase=PHASE_TO_PICKUP },
    { .id=2, .current_floor=1, .busy=0, .state=ELEV_IDLE, .dir=DIR_IDLE,
      .phase=PHASE_TO_PICKUP }
};

static Request_t request_queue[MAX_REQUESTS];
static uint8_t request_count = 0;
```

10.8 Mapping Keypad Input to Floors

The keypad does not directly represent floors in a linear pattern. Therefore, the code maps specific keys to origin floors and destination floors.

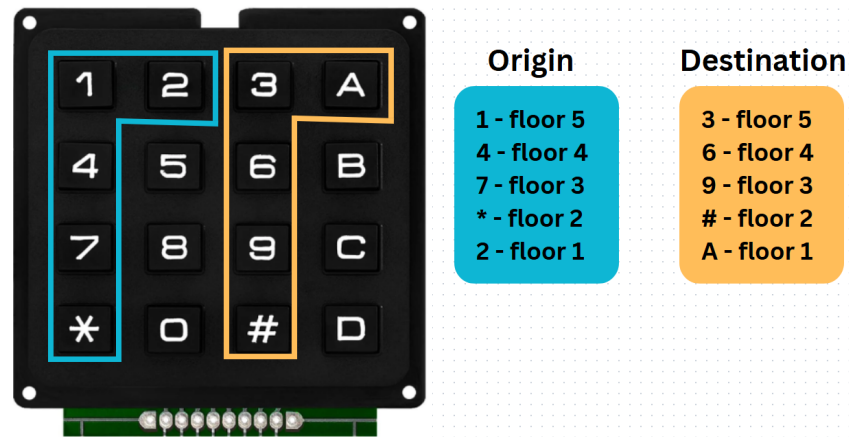


Figure 10.1: Input system

Keypad to Floor Mapping

```
static uint8_t keypad_to_origin(char key)
{
    switch (key) {
        case '1': return 5;
        case '4': return 4;
        case '7': return 3;
        case '*': return 2;
        case '2': return 1;
        default: return 0;
    }
}

static uint8_t keypad_to_destination(char key)
{
    switch (key) {
        case '3': return 5;
        case '6': return 4;
        case '9': return 3;
        case '#': return 2;
        case 'A': return 1;
        default: return 0;
    }
}
```

The mapping divides the keypad into two logical sides: one for selecting origin and the other for selecting destination. Invalid keys return 0, which the main loop treats as “ignore”.

10.9 Request Queue Implementation

Requests are stored in a small FIFO queue. Each request contains an origin floor, destination floor, and the implied direction.

Request Creation and Validation

```
static void add_request(uint8_t origin, uint8_t dest)
{
    if (request_count >= MAX_REQUESTS) {
        printf("[REQ] Queue full, request dropped!\r\n");
        return;
    }
    if (origin < 1 || origin > NUM_FLOORS || dest < 1 || dest >
        NUM_FLOORS || origin == dest) {

        printf("[REQ] Invalid request %d -> %d\r\n", origin, dest);
        return;
    }

    request_queue[request_count].origin = origin;
    request_queue[request_count].destination = dest;
    request_queue[request_count].dir = (dest > origin) ? DIR_UP : DIR_DOWN;

    printf("[REQ] New request: %d -> %d (%s)\r\n",
        origin, dest, dir_str(request_queue[request_count].dir));

    request_count++;
}
```

The function rejects invalid combinations (out of range, or origin equals destination). If valid, it appends to the queue and increments `request_count`.

10.10 Elevator Selection (Greedy Scoring Function)

When an idle elevator is available, the scheduler selects the best elevator for the next pending request.

Elevator Selection Function

```
static int choose_elevator(const Request_t *r)
{
    int best = -1;
    int best_score = 100000;

    for (int i = 0; i < NUM_ELEVATORS; i++) {
        Elevator_t *e = &elevators[i];

        int score = 0;
        if (e->busy) score += 50;
        score += abs((int)e->current_floor - (int)r->origin);
        if (e->busy && e->dir == r->dir) score -= 2;

        if (score < best_score) {
            best_score = score;
            best = i;
        }
    }
    return best;
}
```

The score components work as follows:

- If the elevator is busy, a penalty of 50 is added to strongly prefer idle elevators.
- The absolute distance to the pickup floor is added.
- If the elevator is busy and moving in the same direction as the request, a small bonus is applied by subtracting 2.

In practice, the system only assigns a request if an elevator is idle. Therefore, the direction bonus acts mainly as a stable tie-breaking mechanism for future expansions.

10.11 State Machine for Elevator Motion

The elevator behavior is implemented as a state machine with three states:

- **ELEV_IDLE:** elevator is waiting for work (or waiting for a transition).
- **ELEV_MOVING:** elevator is moving one floor at a time using timing intervals.
- **ELEV_DOORS:** elevator holds position while doors are open for a fixed time.

Additionally, each job is divided into two phases:

- **PHASE_TO_PICKUP:** travel to origin floor.
- **PHASE_TO_DROPOFF:** travel to destination floor.

10.11.1 Starting a Motion Segment

Whenever a job is assigned or a phase changes, the elevator starts moving toward its current target floor.

Starting Movement Toward a Target

```
static void elevator_start_move(Elevator_t *e)
{
    uint8_t target = (e->phase == PHASE_TO_PICKUP)
        ? e->pickup_floor : e->dropoff_floor;

    if (target == e->current_floor) {
        elevator_stop(e->id);
        e->state = ELEV_DOORS;
        e->state_start_time = HAL_GetTick();

        printf("[ARRIVE] Elevator %d already at floor %d\r\n", e->id,
            e->current_floor);
        printf("[DOOR] Elevator %d opening doors\r\n", e->id);
        return;
    }

    e->dir = (target > e->current_floor) ? DIR_UP : DIR_DOWN;
    e->state = ELEV_MOVING;
    e->state_start_time = HAL_GetTick();

    if (e->dir == DIR_UP) elevator_up(e->id);
    else                    elevator_down(e->id);
}
```

```
printf("[MOVE] Elevator %d moving %s (from %d to %d)\r\n",
      e->id, dir_str(e->dir), e->current_floor, target);
}
```

If the elevator is already at the target, it does not move. Instead, it transitions directly to the door state. Otherwise, it selects direction, sets the moving state, records the start time, and commands the servo accordingly.

10.11.2 Assigning Requests to Idle Elevators

The function `assign_requests()` checks if there are pending requests and at least one idle elevator. If so, it pops the first request (FIFO) and assigns it.

Request Assignment to an Elevator

```
static void assign_requests(void)
{
    if (request_count == 0) return;

    int has_idle = 0;
    for (int i = 0; i < NUM_ELEVATORS; i++) {
        if (!elevators[i].busy) { has_idle = 1; break; }
    }
    if (!has_idle) return;

    Request_t r = request_queue[0];
    int idx = choose_elevator(&r);
    if (idx < 0) return;

    Elevator_t *e = &elevators[idx];
    if (e->busy) return;

    for (int i = 0; i < request_count - 1; i++)
        request_queue[i] = request_queue[i + 1];
    request_count--;

    e->pickup_floor = r.origin;
    e->dropoff_floor = r.destination;
    e->phase = PHASE_TO_PICKUP;
    e->busy = 1;

    printf("[ASSIGN] Elevator %d takes request %d -> %d\r\n",
```

```
        e->id, r.origin, r.destination);

    elevator_start_move(e);
}
```

The queue shift operation preserves FIFO ordering. Once assigned, the elevator is marked busy and begins traveling toward pickup.

10.11.3 Periodic Elevator Update

Each elevator is updated repeatedly inside the main loop. The update is non-blocking and uses time differences based on `HAL_GetTick()`.

Elevator State Machine Update

```
static void elevator_update(Elevator_t *e)
{
    uint32_t now = HAL_GetTick();

    if (!e->busy && e->state == ELEV_IDLE) return;

    uint8_t target = (e->phase == PHASE_TO_PICKUP) ? e->pickup_floor
        : e->dropoff_floor;

    if (e->state == ELEV_MOVING) {
        if (now - e->state_start_time >= TIME_PER_FLOOR_MS) {
            e->state_start_time = now;

            if (e->dir == DIR_UP && e->current_floor < NUM_FLOORS)
                e->current_floor++;
            else if (e->dir == DIR_DOWN
                && e->current_floor > 1)
                e->current_floor--;

            printf("[FLOOR] Elevator %d reached floor %d\r\n", e->id,
                e->current_floor);

            if (e->current_floor == target) {
                elevator_stop(e->id);
                e->state = ELEV_DOORS;
                e->state_start_time = now;

                if (e->phase == PHASE_TO_PICKUP) {
```

```

        printf("[PICK] Elevator %d picked passenger
               at floor %d\r\n", e->id,
               e->current_floor);
    } else {
        printf("[DROP] Elevator %d dropped passenger
               at floor %d\r\n", e->id,
               e->current_floor);
    }

    printf("[DOOR] Elevator %d opening doors\r\n", e->id);
}
}
else if (e->state == ELEV_DOORS) {
    if (now - e->state_start_time >= DOOR_TIME_MS) {
        printf("[DOOR] Elevator %d closing doors\r\n", e->id);

        if (e->phase == PHASE_TO_PICKUP) {
            e->phase = PHASE_TO_DROPOFF;
            e->state = ELEV_IDLE;
            e->dir = DIR_IDLE;

            printf("[NEXT] Elevator %d going to destination
                   floor %d\r\n", e->id, e->dropoff_floor);
            elevator_start_move(e);
        } else {
            e->busy = 0;
            e->phase = PHASE_TO_PICKUP;
            e->state = ELEV_IDLE;
            e->dir = DIR_IDLE;

            printf("[IDLE] Elevator %d ready at floor %d\r\n", e->id,
                   e->current_floor);
        }
    }
}
}
}

```

The movement model updates the elevator one floor at a time every `TIME_PER_FLOOR_MS`. When the target is reached, the elevator stops and transitions to `ELEV_DOORS`. After `DOOR_TIME_MS`, the code either transitions to the next phase (pickup → dropoff) or releases the elevator as idle.

10.12 Main Program Flow

10.12.1 Initialization and PWM Startup

The initialization configures HAL, clocks, GPIO, UART, and timers. Then PWM outputs are started and both servos are commanded to their calibrated neutral.

Main Initialization and Safe Startup

```
HAL_Init();
SystemClock_Config();

MX_GPIO_Init();
MX_USART3_UART_Init();
MX_TIM3_Init();
MX_TIM20_Init();

if (HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4) != HAL_OK)
    { Error_Handler(); }
if (HAL_TIM_PWM_Start(&htim20, TIM_CHANNEL_3) != HAL_OK)
    { Error_Handler(); }

elevator_stop(1);
elevator_stop(2);

printf("Elevator system start\r\n");
printf("TIME_PER_FLOOR_MS=%d, DOOR_TIME_MS=%d\r\n",
    TIME_PER_FLOOR_MS, DOOR_TIME_MS);

printf("[CFG] E1 neutral=%u up=%u down=%u deadband=%u\r\n",
    servo_cfg[1].neutral_us, servo_cfg[1].up_us,
    servo_cfg[1].down_us, servo_cfg[1].deadband_us);
printf("[CFG] E2 neutral=%u up=%u down=%u deadband=%u\r\n",
    servo_cfg[2].neutral_us, servo_cfg[2].up_us,
    servo_cfg[2].down_us, servo_cfg[2].deadband_us);
```

This startup sequence is important because continuous rotation servos can begin rotating unexpectedly if the PWM is not initialized to a safe value immediately.

10.12.2 Main Loop Execution

The main loop continuously reads keypad input, builds requests, assigns them when possible, and updates elevator state machines.

Main Loop Logic

```
while (1)
{
    char key = read_keypad();
    static uint8_t origin = 0;

    if (key != 0) {
        if (origin == 0) {
            origin = keypad_to_origin(key);
            if (origin) printf("[INPUT] Origin floor: %d\r\n", origin);
        } else {
            uint8_t dest = keypad_to_destination(key);
            if (dest && dest != origin) {
                printf("[INPUT] Destination floor: %d\r\n", dest);
                add_request(origin, dest);
                origin = 0;
            }
        }
    }

    assign_requests();

    for (int i = 0; i < NUM_ELEVATORS; i++) {
        elevator_update(&elevators[i]);
    }

    HAL_Delay(20);
}
```

The variable `origin` is static so it preserves its value between loop iterations. The user must select an origin first, then a destination. Once both are captured, a request is added to the queue and the system resets `origin` back to zero.

The `HAL_Delay(20)` ensures stable execution timing and prevents excessive keypad polling while keeping the system responsive.

10.13 Timer Configuration Rationale

The PWM signal requires a 20 ms period. Both TIM3 and TIM20 were configured with:

- A prescaler such that the timer counter increments in microseconds.
- A period of 19999, producing a 20,000 μ s cycle.

This makes the compare register directly equal to pulse width in microseconds, simplifying the servo control functions.

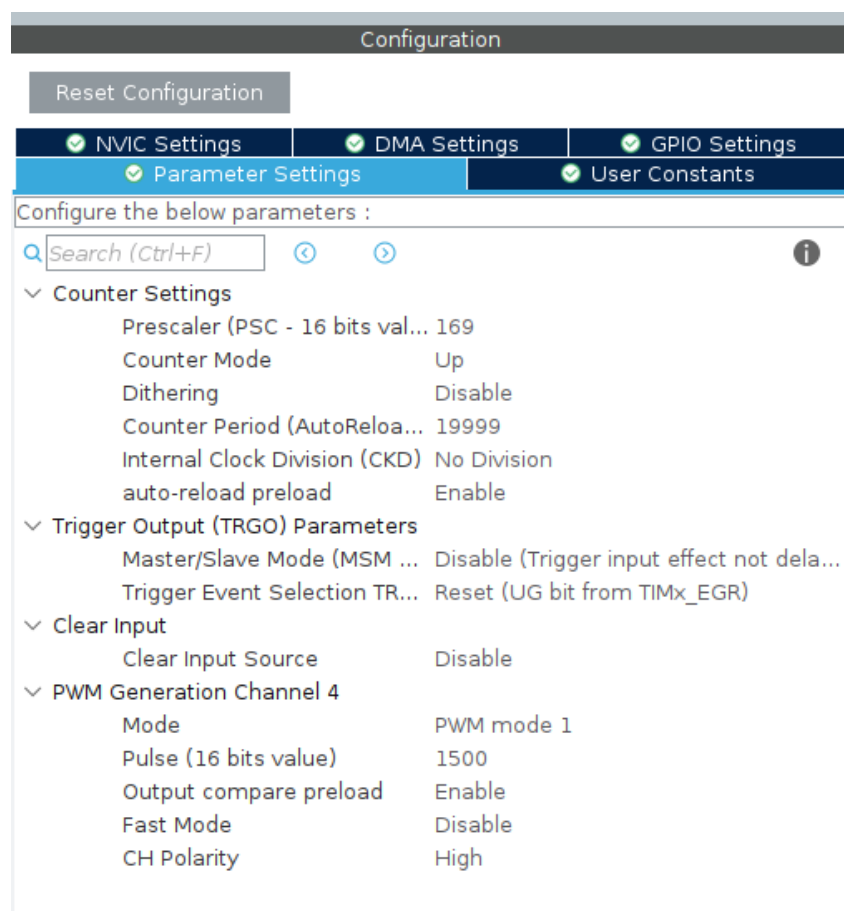


Figure 10.2: Timer Configuration

Chapter 11

Phase 4; Results

In this section, a video is provided to clearly demonstrate the final system behavior and results, including the elevator motion, scheduling response, and overall hardware integration performance.

- **Results:** https://youtu.be/PxLS98n_gyU?si=1XM93507TXocclA

Chapter 12

Conclusion

This project successfully demonstrated the complete design, implementation, and integration of a dual-elevator control system using an STM32-based embedded platform. The work combined mechanical design, hardware integration, and software development into a cohesive system capable of responding to user inputs and executing coordinated elevator scheduling and motion control.

From a mechanical perspective, the elevator structure and cabins were designed in SolidWorks with careful consideration of dimensional accuracy, structural rigidity, and 3D printing constraints. The use of customized tolerances and reinforced base elements resulted in a stable physical platform that supported reliable operation of the elevator mechanism.

On the hardware side, the integration of the STM32 NUCLEO-G474RE, continuous-rotation servomotors, matrix keypad, and external power supply was carried out following good embedded systems practices. Proper power distribution, common grounding, and peripheral configuration ensured stable electrical behavior and reproducible results during testing.

The software implementation represented the core intelligence of the system. A structured, non-blocking scheduling algorithm was developed to manage multiple elevator requests, assign tasks efficiently, and simulate realistic elevator behavior through state machines and time-based transitions. The modular code structure, combined with detailed debugging output via UART, enabled clear observation of system behavior and simplified validation and tuning.

Overall, this project provided valuable hands-on experience in embedded system design, real-time control, and system-level integration. The final prototype demonstrates a functional and extensible elevator control system that could be further expanded with additional features such as sensor feedback, advanced scheduling strategies, or closed-loop motion control.

Bibliography

- [1] C. Creator, “How to make a digital lock using keypad and arduino,” 2021. Accessed: 2025-10-12.
- [2] Chandong83, “Rfid tutorial and example with rc522 and arduino,” 2017. Accessed: 2025-10-12.
- [3] B. Wireless, “Understanding sensor interfaces: Uart, i2c, spi, and can,” 2022. Accessed: 2025-10-12.
- [4] STMicroelectronics, “Nucleo-g474re development board,” 2025. Accessed: 2025-10-12.
- [5] AntPlusTech, “Wi-fi and bluetooth communication with stm32 and esp modules,” 2018. Accessed: 2025-10-12.
- [6] Instructables, “Getting started with the esp8266 (esp-01),” 2019. Accessed: 2025-10-12.